Simple Command Sequencer Generator

Autocoder User Guide

1 Description

The purpose of this generator is to provide a user friendly way of creating packets formed from a list of data products. The generator takes a YAML model file as input which specifies the packets to produce, the data products to put in each packet, the period that the packet will be emitted at, and whether the packet is enabled or disabled on startup. From this information, the generator autocodes an Ada specification file which contains a data structure that should be passed to the Product Packetizer component upon initialization.

Note the example shown in this documentation is used in the unit test of this component so that the reader of this document can see it being used in context. Please refer to the unit test code for more details on how this generator can be used.

2 Schema

The following pykwalify schema is used to validate the input YAML model. Model files must be named in the form *optional_name.assembly_name.product_packets.yaml* where *optional_name* is the specific name of this set of packets and is only necessary if there is more than one Product Packetizer component instance in an assembly. The *assembly_name* is the assembly which these product packets will be used in, and the rest of the model file name must remain as shown. Generally this file is created in the same directory or near to the assembly model file. The schema is commented to show what each of the available YAML keys are and what they accomplish. Even without knowing the specifics of pykwalify schemas, you should be able to glean some knowledge from the file below.

```
# Schema for command_sequences YAML files
# This defines the structure and validation rules

type: map
mapping:
  name:
    required: true
    type: str
    desc: |
      The name of the command sequences package. This will be used as the
      Ada package name, so it must follow Ada naming conventions.

  description:
    required: false
    type: str
    desc: |
      Description of this command sequences package. This will appear in
      the generated Ada code comments.

  preamble:
    required: false
    type: str
    desc: |
```

```
    Custom preamble text to include in the generated Ada package.

sequences:
  required: true
  type: seq
  desc: |
    List of command sequence definitions. Each sequence defines a named
    command that executes a series of sub-commands.
  sequence:
    - type: map
      mapping:
        name:
          required: true
          type: str
          desc: |
            Name of this sequence. This will become a command that can be
            executed. Must follow Ada naming conventions.

        description:
          required: false
          type: str
          desc: |
            Description of what this sequence does. Will appear in generated
            code and documentation.

        arg_type:
          required: false
          type: str
          desc: |
            The Ada type for this sequence's argument. If not specified,
            the sequence takes no arguments. Must be a valid Ada type name,
            possibly qualified (e.g., "Power_Mode.T" or "Packed_U16.T").

        wait_for_command_completion:
          required: false
          type: bool
          default: true
          desc: |
            Default setting for whether to wait for each sub-command to
            complete before executing the next one. Individual steps can
            override this.

        continue_on_failure:
          required: false
          type: bool
          default: false
          desc: |
            If true, the sequence will continue executing even if a
            ↪  sub-command
            fails. If false, the sequence will abort on the first failure.

        sequence:
          required: true
          type: seq
          desc: |
            The ordered list of sub-commands to execute. Each step defines
            a command to send and optionally an argument expression.
          sequence:
            - type: map
              mapping:
                command:
```

```
                      required: true
                      type: str
                      desc: |
                        The command to execute in the format
                        ↪ "Component.Command_Name".
                        The component must be a valid component instance name in
                        ↪ your
                        assembly, and the command must exist on that component.

                    arg:
                      required: false
                      type: str
                      desc: |
                        An Ada expression that evaluates to the argument for this
                        command. You can reference the parent sequence's argument
                        using "Arg.field_name" syntax. If the command takes no
                        argument, omit this field.

                        Examples:
                          - "Arg" - pass through the entire sequence argument
                          - "Arg.Power_Mode" - pass a field from the sequence
                          ↪ argument
                          - "(Mode => Arg.Mode, Enable => True)" - construct a
                          ↪ record
                          - "42" - a literal value

                    wait_for_completion:
                      required: false
                      type: bool
                      desc: |
                        Whether to wait for this specific command to complete
                        ↪ before
                        proceeding. If not specified, uses the parent sequence's
                        wait_for_command_completion setting.

# Additional validation rules (enforced in the model Python code):
# - Command format must be "Component.Command" with valid identifiers
# - Referenced types in arg_type must exist in the system
# - Argument expressions must be valid Ada syntax
# - Component and command names must match instances in the assembly
```

3 Example Input

The following is an example product packet input yaml file. Model files must be named in the form *optional_name.assembly_name.product_packets.yaml* where *optional_name* is the specific name of the product packets and is only necessary if there is more than one Product Packetizer component instance in an assembly. The *assembly_name* is the assembly which these packets will be used in, and the rest of the model file name must remain as shown. Generally this file is created in the same directory or near to the assembly model file. This example adheres to the schema shown in the previous section, and is commented to give clarification.

```
# Command Sequences Definition
# This file defines composite commands that execute sequences of sub-commands

name: Flight_Sequences
description: Common flight software command sequences

sequences:
```

```yaml
  # Simple sequence with no arguments
  - name: Sequence_A
    description: Execute commands on component A
    wait_for_command_completion: true
    continue_on_failure: true

    sequence:
      - command: Component_A.Command_1
      - command: Component_A.Command_2
        arg: (Seconds => 3, Subseconds => 14)
      - command: Sleep
        arg: (Seconds => 3, Milliseconds => 0)
      - command: Component_A.Command_3
        arg: (Value => 3)

  - name: Sequence_B
    description: Perform generic arguments with component A and B
    wait_for_command_completion: true
    continue_on_failure: false
    arg_type: Packed_U32.T

    sequence:
      - command: Component_A.Command_3
        arg: Arg
      - command: Component_B.Command_3
        arg: Arg

  - name: Sequence_C
    description: No-wait sequence for direct-dispatch testing
    wait_for_command_completion: false
    continue_on_failure: false

    sequence:
      - command: Component_A.Command_1
      - command: Component_A.Command_1

  - name: Sequence_D
    description: Sequence with a large sleep value to test an overflow sleep
    wait_for_command_completion: true
    continue_on_failure: false

    sequence:
      - command: Sleep
        arg: (Seconds => 30000, Milliseconds => 0)
```

4 Example Output

The example input shown in the previous section produces the following Ada output. The `Packet_List` variable should be passed into the Product Packetizer component's discriminant during assembly initialization.

The main job of the generator in this case was to verify the input YAML packets for validity and then to translate the data to an Ada data structure for use by the component.

```ada
-- Auto-generated by Adamant command sequences generator.
-- Do not edit by hand.
-- Common flight software command sequences
with Simple_Sequencer_Types; use Simple_Sequencer_Types;
with Test_Assembly_Commands; use Test_Assembly_Commands;
```

```
with Scs_Arg_Resolver;
with Command_Types;
with System;
with Sequence_Arg_Utils; use Sequence_Arg_Utils;
with Sys_Time_32;
with Packed_U32;
package Test_Assembly_Command_Sequences_Example_Sequences is
   ----------------------------------------------------------------------------
   -- Resolver types - one per dynamic step, each encodes a traversal path.
   ----------------------------------------------------------------------------

   type Sequence_B_Step_0_Resolver_T is new Scs_Arg_Resolver.T with null record;

   overriding function Resolve (
      Resolver : Sequence_B_Step_0_Resolver_T;
      Data     : System.Address
   ) return Command_Types.Command_Arg_Buffer_Type;

   Sequence_B_Step_0_Resolver : aliased constant
      Sequence_B_Step_0_Resolver_T := (null record);


   type Sequence_B_Step_1_Resolver_T is new Scs_Arg_Resolver.T with null record;

   overriding function Resolve (
      Resolver : Sequence_B_Step_1_Resolver_T;
      Data     : System.Address
   ) return Command_Types.Command_Arg_Buffer_Type;

   Sequence_B_Step_1_Resolver : aliased constant
      Sequence_B_Step_1_Resolver_T := (null record);

   ----------------------------------------------------------------------------
   -- Step arrays - one constant per sequence.
   ----------------------------------------------------------------------------
   Sequence_A_Steps : aliased constant Step_Array :=
     [
      0 => (Kind       => Command_Step,
                          Id         => Component_A_Command_1,
                          Arg_Length => 0,
                          Arg        => [others => 0]),
      1 => (Kind       => Command_Step,
                          Id         => Component_A_Command_2,
                          Arg_Length =>
                           ↪  Sys_Time_32.Serialization.Serialized_Length,
                          Arg        => To_Arg
                           ↪  (Sys_Time_32.Serialization.To_Byte_Array
                           ↪  (((Seconds => 3, Subseconds => 14))))),
      2 => (Kind       => Sleep,
                          Id         => 0,
                          Arg_Length => 0,
                          Sleep_Arg  => (Seconds => 3, Milliseconds => 0)),
      3 => (Kind       => Command_Step,
                          Id         => Component_A_Command_3,
                          Arg_Length =>
                           ↪  Packed_U32.Serialization.Serialized_Length,
                          Arg        => To_Arg
                           ↪  (Packed_U32.Serialization.To_Byte_Array (((Value
                           ↪  => 3)))))
     ];
   Sequence_B_Steps : aliased constant Step_Array :=
```

```
      [
       0 => (Kind       => Dynamic_Command_Step,
                           Id         => Component_A_Command_3,
                           Arg_Length =>
                            ↪  Packed_U32.Serialization.Serialized_Length,
                           Resolver => Scs_Arg_Resolver.T_Access'(Sequence_B_St
                            ↪  ep_0_Resolver'Access)),
       1 => (Kind       => Dynamic_Command_Step,
                           Id         => Component_B_Command_3,
                           Arg_Length =>
                            ↪  Packed_U32.Serialization.Serialized_Length,
                           Resolver => Scs_Arg_Resolver.T_Access'(Sequence_B_St
                            ↪  ep_1_Resolver'Access))
      ];
   Sequence_C_Steps : aliased constant Step_Array :=
      [
       0 => (Kind       => Command_Step,
                           Id         => Component_A_Command_1,
                           Arg_Length => 0,
                           Arg        => [others => 0]),
       1 => (Kind       => Command_Step,
                           Id         => Component_A_Command_1,
                           Arg_Length => 0,
                           Arg        => [others => 0])
      ];
   Sequence_D_Steps : aliased constant Step_Array :=
      [
       0 => (Kind       => Sleep,
                           Id         => 0,
                           Arg_Length => 0,
                           Sleep_Arg  => (Seconds => 30000, Milliseconds => 0))
      ];
   ------------------------------------------------------------------------------
   -- Sequences_Table
   ------------------------------------------------------------------------------
   Sequences_Table : aliased constant Sequences_Type :=
      [
       0 => (
          Wait_For_Cmd_Resp   => True,
          Abort_On_Failed_Cmd => False,
          Steps               => Sequence_A_Steps'Access),
       1 => (
          Wait_For_Cmd_Resp   => True,
          Abort_On_Failed_Cmd => True,
          Steps               => Sequence_B_Steps'Access),
       2 => (
          Wait_For_Cmd_Resp   => False,
          Abort_On_Failed_Cmd => True,
          Steps               => Sequence_C_Steps'Access),
       3 => (
          Wait_For_Cmd_Resp   => True,
          Abort_On_Failed_Cmd => True,
          Steps               => Sequence_D_Steps'Access)
      ];

   Sequences : constant Sequences_Access := Sequences_Table'Access;

end Test_Assembly_Command_Sequences_Example_Sequences;
```

```
-- Auto-generated by Adamant command sequences generator.
-- Do not edit by hand.
with Packed_U32;
package body Test_Assembly_Command_Sequences_Example_Sequences is

   overriding function Resolve (
      Resolver : Sequence_B_Step_0_Resolver_T;
      Data     : System.Address
   ) return Command_Types.Command_Arg_Buffer_Type is
      Input : Packed_U32.T
         with Import, Address => Data;
   begin
      return To_Arg (
         Packed_U32.Serialization.To_Byte_Array (
            Input
         )
      );
   end Resolve;

   overriding function Resolve (
      Resolver : Sequence_B_Step_1_Resolver_T;
      Data     : System.Address
   ) return Command_Types.Command_Arg_Buffer_Type is
      Input : Packed_U32.T
         with Import, Address => Data;
   begin
      return To_Arg (
         Packed_U32.Serialization.To_Byte_Array (
            Input
         )
      );
   end Resolve;

end Test_Assembly_Command_Sequences_Example_Sequences;
```

5 Special Items

The Product Packetizer allows you to specify “special" items to include in a packet that reflect internal data of the Product Packetizer component itself. Currently, the only supported “special" items are packet periods of the packets produced by the Product Packetizer. Packet 5, specified above, includes these items by specifying a data product within the Product Packetizer, ie. `Product_Packetizer_Instance.Packet_4_Period`. The Product Packetizer doesn't actually have any data products, so this nomenclature instead denotes a special item. In this case, we want to include the current packet period value (a 4 byte unsigned integer) for Packet 4 into the packet. A period can be specified for any packet included in the YAML model using this pattern. Error checking at the modeling level will prevent you from specifying a packet period for a packet that does not exist.